Evaluating HPK for Running Cloud-Native Workloads on Slurm Clusters

Antony Chazapis
Institute of Computer Science
Foundation for Research and Technology - Hellas (FORTH)
Heraklion, Crete, Greece
chazapis@ics.forth.gr

Lefteris Vassilakis
Institute of Computer Science
Foundation for Research and Technology - Hellas (FORTH)
Heraklion, Greece
vassilakis@ics.forth.gr

Giannis Petsis
Institute of Computer Science
Foundation for Research and Technology - Hellas (FORTH)
Heraklion, Greece
Computer Science Department
University of Crete
Heraklion, Greece
petsis@ics.forth.gr

Manolis Marazakis
Institute of Computer Science
Foundation for Research and Technology - Hellas (FORTH)
Heraklion, Greece
maraz@ics.forth.gr

Angelos Bilas
Institute of Computer Science
Foundation for Research and Technology - Hellas (FORTH)
Heraklion, Greece
Computer Science Department
University of Crete
Heraklion, Greece
bilas@ics.forth.gr

Abstract

The escalating complexity of applications and services encourages a shift towards higher-level data processing pipelines that integrate both Cloud-native and HPC steps into the same workflow. Cloud providers and HPC centers typically provide both execution platforms on separate resources. In this paper we explore a more practical design that enables running unmodified Cloud-native workloads directly on the main HPC cluster, avoiding resource partitioning and retaining the HPC center's existing job management and accounting policies.

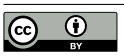
CCS Concepts

• **Computer systems organization → Cloud computing**; *Parallel architectures*; • **Software and its engineering → Cloud computing**; *Interoperability*.

Keywords

Cloud-HPC convergence, Kubernetes, Virtual Kubelet, Slurm, Singularity, Apptainer

ACM Reference Format:
Antony Chazapis, Lefteris Vassilakis, Giannis Petsis, Manolis Marazakis, and Angelos Bilas. 2025. Evaluating HPK for Running Cloud-Native Workloads on Slurm Clusters. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25), November 16–21, 2025, St Louis, MO, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3731599.3767352



This work is licensed under a Creative Commons Attribution 4.0 International License.
SC Workshops '25, St Louis, MO, USA
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1871-7/25/11
https://doi.org/10.1145/3731599.3767352

1 Introduction

Cloud-HPC convergence for Big Data processing pipelines that combine Cloud-native with HPC steps is most often realized with interfacing mechanisms for submitting HPC jobs from the Cloud side or vice versa. However, bridging separate Cloud and HPC environments has proven to be challenging, as it involves synchronizing data between sites and coping with access restrictions. Having two separate setups also elevates the associated hardware and maintenance costs.

In this paper, we explore an HPC-centric solution that accommodates both Cloud and HPC software stacks on the same physical resources. We focus our work on Kubernetes [13], currently the most prominent distributed container orchestrator for supporting the "Cloud-native" ecosystem [3]. We first present *HPK* (the name implies the ability to run Kubernetes in an HPC environment), an open-source integration of unmodified Kubernetes components and custom modules that runs as a user-level service, requiring minimal support from the underlying HPC system software [7, 26]. With HPK, users run their own private "mini Clouds" as in-cluster applications that delegate scheduling decisions to Slurm [20] and execution to the Singularity/Apptainer container runtime [1, 22, 29], to comply with organization policies and established resource accounting mechanisms. We then evaluate HPK by running unmodified Cloud-native computational and machine learning workloads in an HPC cluster, showcasing how it enables users to tap on the vast collection of available data processing frameworks, both as

stand-alone solutions and in hybrid computation scenarios alongside HPC codes. Additionally, we compare the runs with the same codes running natively on Kubernetes using the same hardware.

2 Related work

We classify work related to Cloud-HPC convergence in two main categories: Systems that maintain the *separation* of Cloud and HPC, and systems that *embed* one resource management framework into the other. In the former case there are two separate resource managers, while in the latter there is a single authority that controls hardware allocations, shared by both Cloud and HPC deployments; the embedded framework delegates resource management decisions to the overall cluster manager. Works that assume separate clusters can further be divided into *bridging* solutions that operate within the context of the Cloud or HPC runtime, allowing the transparent submission of remote jobs, or *third-party* systems that operate in their own context and are able to administer tasks in both remote Cloud and HPC installations.

Many bridging solutions are available for Kubernetes, enabling Cloud users to integrate the execution of remote HPC tasks into their workflows. The *hpc-connector* [30] uses a custom container that when executed as part of a Kubernetes Job, will forward work to the HPC cluster, track its execution, and collect any results. Other tools use Kubernetes Custom Resource Definitions (CRDs) that determine how to describe jobs targeted for the HPC cluster. *Torque-Operator* [35] interfaces a Kubernetes installation to a Torque-based HPC cluster, while the *Bridge Operator* [31] offers a wider compatibility of remote job execution facilities. To avoid CRDs, KNoC (Kubernetes Node on HPC Cluster) [32] implements a virtual node for Kubernetes that transparently manages the container lifecycle on a remote HPC cluster using Slurm and Singularity. This technique effectively allows users to employ existing Cloud-native tools, such as Argo Workflows [2] to express complex data-processing pipelines for both Cloud and HPC without explicit remote execution steps.

Bridging solutions are especially useful when Cloud and HPC resources are colocated. HPC centers increasingly support on-demand provisioning of Cloud resources—even as partitions of the main HPC machine. However, when the two are remotely situated, bridging suffers from the overhead of maintaining data copies. The user must prepare and send inputs to the remote HPC cluster before issuing any tasks, and then place back outputs in the Kubernetes context. Data synchronization in hybrid workflows is addressed by StreamFlow [27], a third-party system which extends the workflow language with declarative descriptions of execution sites (either Cloud or HPC) and their relationship to workflow nodes. The runtime automatically infers data dependencies, so to copy required data where needed before running each step.

Embedded convergence solutions avoid data copies and the requirement to manage and maintain two separate setups, as both Cloud and HPC share a common hardware platform. In a Kubernetes cluster, the MPI Operator [15] defines the `MPIJob` Kubernetes CRD, which is realized by running a group of ephemeral container instances that include the MPI process. *Virtual clusters* [34] take this notion a step further, as the execution containers embed a Slurm deployment (controller and workers) for compatibility with existing scripts. Additionally, the Slurm controller is extended with a custom protocol that requests resources from the Kubernetes scheduler, maintaining Kubernetes's role of managing the whole infrastructure. A custom Kubernetes scheduler allows applying different container placement policies for *HPC* and *data center* services (Kubernetes deployments that run in other containers) with a goal to minimize interference effects. Soperator [21] is based on a similar idea, allowing Kubernetes users to define `SlurmCluster` resources, which are then realized by the operator using pods (a combination of *Login*, *Controller*, and *Worker* pods that collectively form the Slurm environment). In addition, Soperator mounts a shared filesystem across all pods of the same Slurm cluster, in an effort to align the usage experience to the one of a physical cluster. Slurm-operator [19] (part of SchedMD's Slinky projects [17]) can manage both a containerized Slurm deployment that operates totally inside Kubernetes, or a *hybrid* setup where some components (i.e., compute workers) run on bare-metal. This practically allows to use Kubernetes on a Cloud provider to autoscale an existing Slurm setup. SUNK [8] (SlUrm oN Kubernetes) uses a cluster-wide Slurm deployment that is integrated into Kubernetes as a specialized scheduler. External Slurm *login nodes* provide users with the typical interface of running Slurm scripts, while actual jobs are submitted to the Kubernetes cluster nodes. A *syncer* plugin monitors resource utilization to synchronize state information between Kubernetes and Slurm.

Embedded configurations have also been classified as [33]: *Over*, where Slurm is in control of the cluster, creating Kubernetes environments ephemerally within batch jobs, *adjacent*, when both Slurm and Kubernetes are installed on the same physical nodes but share a common scheduler (i.e., Kubernetes uses Slurm to place jobs [18]), and *under*, when Slurm-enabled pods are deployed in Kubernetes (such as many of the systems described in the previous paragraph). HPK falls within the *over* class, however it does not create a full Kubernetes environment as a batch job (or as a dynamic partition, as may be possible using Flux child instances [25]), but rather transforms each Kubernetes deployment to an individual Slurm script, allowing for better scheduling flexibility and finer-grain resource sharing.

We are not aware of any other system that embeds Kubernetes in HPC in such a way. Usernetes [23] has unintentionally been exploring this direction, providing a Kubernetes distribution that can run without root privileges. Initial versions were, however, limited to a single node, so HPK followed a different approach to provide working container communication across the physical cluster. The current—second generation—of Usernetes utilizes a container-level distributed networking stack. We are currently evaluating this option as an alternative to HPK's solution that involves using some CNI plugin at the Singularity/Apptainer level.

3 Design & implementation

We aim to establish a framework that enables HPC users to execute Kubernetes workloads within a standard cluster environment. From a design perspective, the requirements are:

- *Compatibility*—To support the majority of Kubernetes applications and services, all native language abstractions, such as pods (one or more containers that are scheduled and scaled

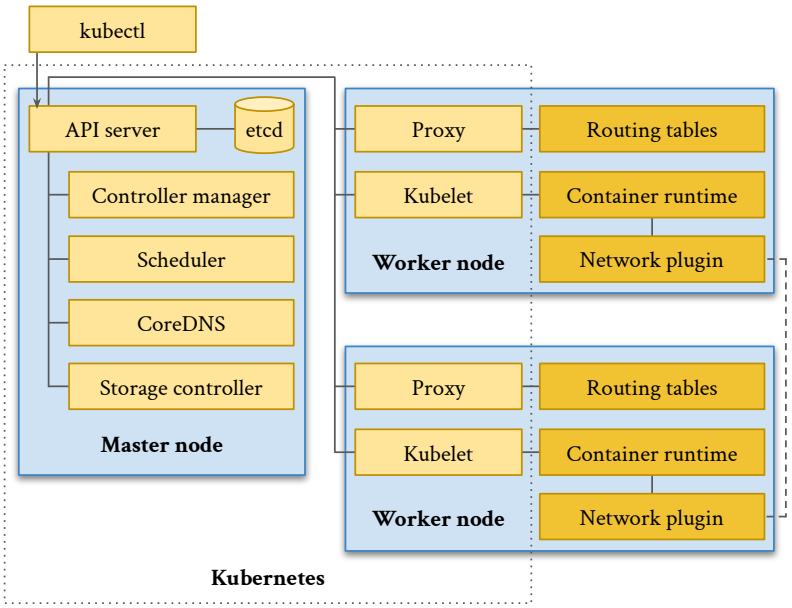


Figure 1: Components involved in a typical Kubernetes deployment on bare-metal.

as a group), deployments, services, jobs, and volumes should be available and fully functional. Exceptions can only include constructs that directly relate to physical hardware resources (i.e., "NodePort" services that request a specific port number for exposing services at Kubernetes nodes). Microservices require that pods are individually addressable, supporting inter-container networking and internal service discovery.

- *Compliance*—All resource management decisions should be delegated to the cluster manager operating the cluster (i.e., Slurm). Organization policies for resource allocation and accounting should be fully respected. Running workloads should be visible at the level of the cluster manager. Also, minimal configuration changes should be required to be done at the host level by HPC administrators. Reliance on special libraries or binaries that execute with "elevated" permissions should be avoided.
- *Usability*—Make it easy for users to deploy. All binaries should be packaged up with their dependencies into a container, with no host-specific requirements. All configuration should reside in the user's home directory.

Kubernetes is implemented as a set of communicating subsystems that collectively provide the functionality of distributed container orchestration. A typical deployment constitutes of the following (Figure 1):

- *API server*—The "heart" of Kubernetes. The main interface to the cluster and the synchronization point for all controllers.
- *etcd*—The key-value store holding all state. Always accessed through the API server.
- *Controller manager*—Watches for configuration changes or failures and performs all necessary actions to reach the desired state set by the user. The controller manager includes the controllers (or *operators*) that implement the logic for the base Kubernetes abstractions. As all controllers, it communicates only with the API server.
- *Scheduler*—A controller that decides which node will be used to run new pods.
- *CoreDNS*—A controller and DNS server for implementing naming and discovery for pods and internal cluster services.
- *Kubelet*—An agent running on each worker node, implementing the pod lifecycle using a specific container runtime (i.e., containerd).
- *Network plugin*—A service supporting the Container Network Interface (CNI) specification that assigns addresses to pods. The network plugin, which—depending on the Kubernetes version—is used by the kubelet or the container runtime directly, implements the Kubernetes network model [11]. In addition to assigning unique, cluster-wide addresses, it makes sure that pods can communicate with each other across hosts. It may also realize traffic shaping policies or other network-level features.
- *Network proxy*—Creates local network routes for virtual IP addresses used by "ClusterIP" services. Runs on each worker node, alongside the kubelet.
- *Storage controller*—A controller that provisions storage of some type that can be attached to pods. Creates physical volumes to match requested persistent volume claims. While this controller is optional, we consider it a core component for a functioning system.

From these components, the *API server*, *etcd*, *controller manager*, and *CoreDNS* are self-contained services that can run as user processes. They are used unmodified in HPK, so from a user's perspective HPK provides the same frontend interface and API as any

other Kubernetes deployment. Existing configurations using advanced language features, like isolated deployments in different namespaces and access control (RBAC) rules, can be directly applied without changes.

To interface with the container runtime, HPK implements a custom *kubelet*. *hpk-kubelet* layers above both Slurm and Singularity/Apptainer for execution, translating the Kubernetes-level container lifecycle to Slurm scripts containing Singularity/Apptainer commands, while also synchronizing Kubernetes with the respective Slurm-level job states (i.e., enqueued jobs are marked as "pending" pods in Kubernetes, "running" when started, or "failed" if they produce errors). With a single kubelet proxying requests to Slurm, the whole HPK integration can be visualized as a *translation service*: Workloads enter in YAML format through the Kubernetes API endpoint and exit as Slurm scripts from hpk-kubelet (Figure 2). We use generic Slurm directives in scripts that are not tied to a specific Slurm version. This design satisfies the *compliance* requirement, as individual Kubernetes workloads transparently show up in Slurm queues. Moreover, Kubernetes-level resource requests (i.e., number of CPUs, amount of memory, or even GPUs) are forwarded directly to Slurm, as are some HPK-specific pod annotations that can be used to further customize execution. hpk-kubelet is implemented as a *Virtual Kubelet Provider* [24]. To respect pod network semantics (containers within the same pod share the same external IP address and can use *localhost* to communicate with each other internally), it uses an embedded container topology: hpk-kubelet starts a "parent" container with an HPK-specific agent, which in turn runs and manages each container of the pod (the agent has replaced the pod execution mechanics of earlier versions based on bash scripts). The pod IP address is assigned to the parent container; "child" containers run within the same network context without extra IP addresses.

In an HPK deployment, the Kubernetes cluster consists of just one worker node that represents the total amount of computing resources available to the user. Since cluster-level scheduling is to be performed by Slurm, HPK employs a custom, simplified *pass-through scheduler* that makes no scheduling decisions, but always selects hpk-kubelet to run workloads.

For pod and service networking, HPK avoids the *network plugin* and *network proxy* subsystems, as they perform actions at the system level as the root user. Pod addresses should actually be assigned by the container runtime. Singularity/Apptainer supports CNI plugins and can be easily set up to delegate network addressing to a cluster-wide service, which must be set up by system administrators at the node level (i.e., Flannel [6]). This, in addition to allowing containers to run as *fakeroot* for supporting common Docker images that use the root user, are the only changes HPK currently requires from the HPC environment; both being configuration options of the container runtime.

To avoid the network proxy, HPK completely disables "ClusterIP" services, via a Kubernetes *admission controller*—a hook that monitors API requests and may reject or mutate them before reaching the API server. In Kubernetes, services can explicitly request to not use a virtual service IP (also called "headless" services). In such cases, service discovery continues to function, as CoreDNS maps the service name to the actual pod IPs instead of the virtual service address. Thus, microservice architectures are not affected by the lack of service-specific IPs. If load-balancing between pods is necessary, it can be implemented by using an additional service within a deployment.

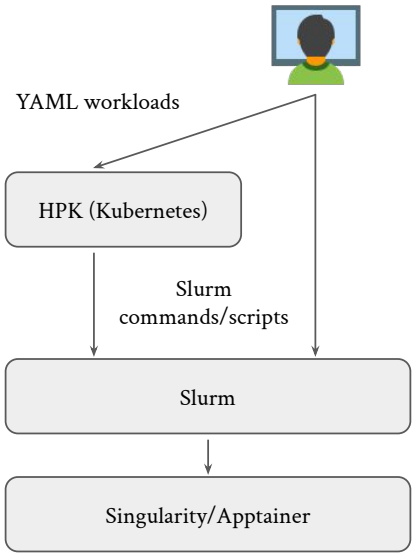


Figure 2: HPK translates Kubernetes workloads to Slurm and Singularity/Apptainer.

For storage, HPK supports "HostPath" volumes—Kubernetes volumes that are bound to existing directories on the host. These can then be used by existing *storage controllers*, such as OpenEBS [16], to provide container storage from different local or cluster-wide facilities through separate storage classes. As an example, users may deploy one OpenEBS storage class over node-local NVMe devices for temporary data, and another over their Lustre-backed home directory. Higher-level storage services, such as object stores, can then utilize these storage classes.

The overall architecture of HPK is shown in Figure 3. The *control plane container* (in white background) includes all HPK components except the hpk-kubelet. It is currently implemented as a custom integration script that packs official releases of the API server, etcd, controller manager, and CoreDNS binaries (all corresponding to a specific Kubernetes version), along HPK's pass-through scheduler and service admission controller. At runtime, it generates all necessary internal keys and certificates, bootstraps the Kubernetes control plane by initializing the executables in order, and produces the configuration file containing the endpoint and credentials needed to connect to the API server. Then, the hpk-kubelet uses this configuration to connect and announce its availability as a node. An experimental version of HPK implements most of the control plane container components with K3s [9], a lightweight Kubernetes distribution that packages all subsystems into a single binary. Both the control plane container and hpk-kubelet run via Slurm with minimal resource requirements on any cluster node. Also, the control plane can be configured to resume its state in case it fails or expires due to exceeding the time limit imposed by the HPC site.

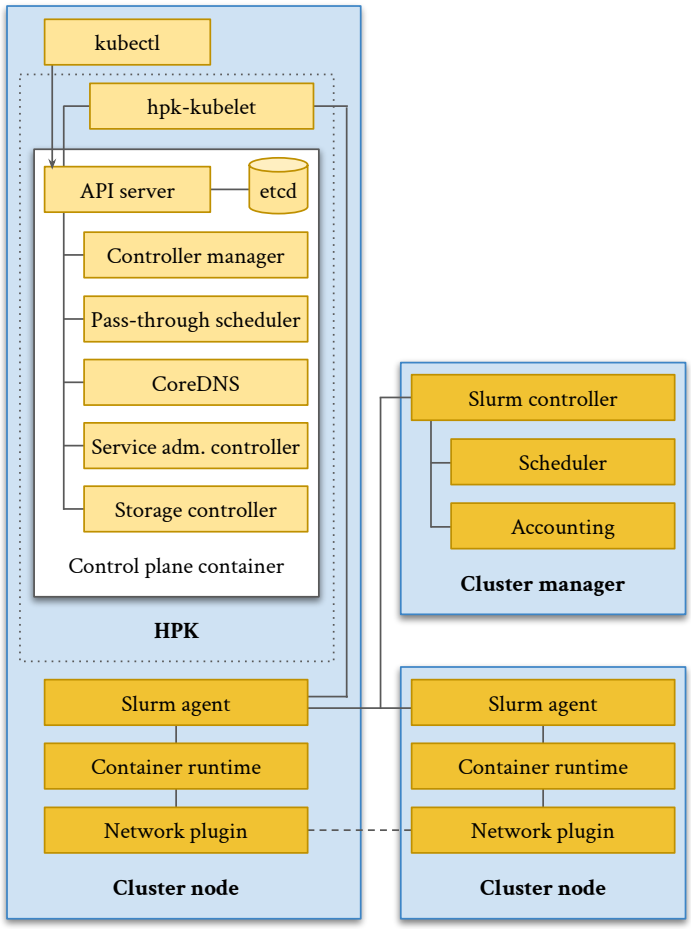


Figure 3: HPK architecture.

4 Evaluation

HPK successfully runs several applications using Cloud-native frameworks without modifications, including the Spark Operator [12], Dask [4], and Kubeflow Training Operator [10]. We have also used HPK to run containerized workflows using Argo Workflows with placement of artifacts on MinIO [14] (HPK can run all workflow examples that are included in the Argo Workflows repository, supporting all language features).

To evaluate HPK's efficiency to deploy unaltered Cloud-native workloads on HPC, we use AWS's ParallelCluster, which provides a Cloud-based HPC-as-a-service environment managed by Slurm. For benchmarking, we compare this setup against an Amazon EKS cluster under identical hardware configurations. Both clusters consist of 8 c5.4xlarge nodes, each with 16 vCPUs, 32 GiB memory, and up to 10 GiB network bandwidth. We leverage the `OnNodeConfigured` action of ParallelCluster's configuration to execute a custom script upon node initialization in order to install additional software required by HPK. This includes Apptainer with the Flannel CNI plugin and Flannel, a tool to distribute private IPs to container instances and manage routes across nodes. Apptainer is set up to use Flannel for networking when running fakeroot containers (containers that internally use the root user), which is the default HPK configuration to allow seamless execution of Docker images.

For each experiment, we connect as a non-root user to the cluster's login node and run both HPK's control plane container, as well as hpk-kubelet. By setting the `KUBECONFIG` environment variable to the configuration file produced, we can interface with HPK using common tools, such as `kubectl` and `helm`.

4.1 Spark TPC-DS

TPC-DS is an industry standard benchmark for measuring the performance of data processing systems. It consists of 99 queries of varying complexity, covering multiple aspects of a system's capabilities. We run a sample integration provided by Amazon for EKS [5], which uses Spark SQL orchestrated via the Spark Operator. Our goal is to evaluate how HPK handles Spark-based workloads as they would typically be deployed in the Cloud. The operator streamlines the deployment and management of Apache Spark applications on Kubernetes by defining the `SparkApplication` CRD. It handles the entire lifecycle of execution, including submission, scaling, and cleanup, and provides logging and monitoring mechanisms for enhanced visibility into performance.

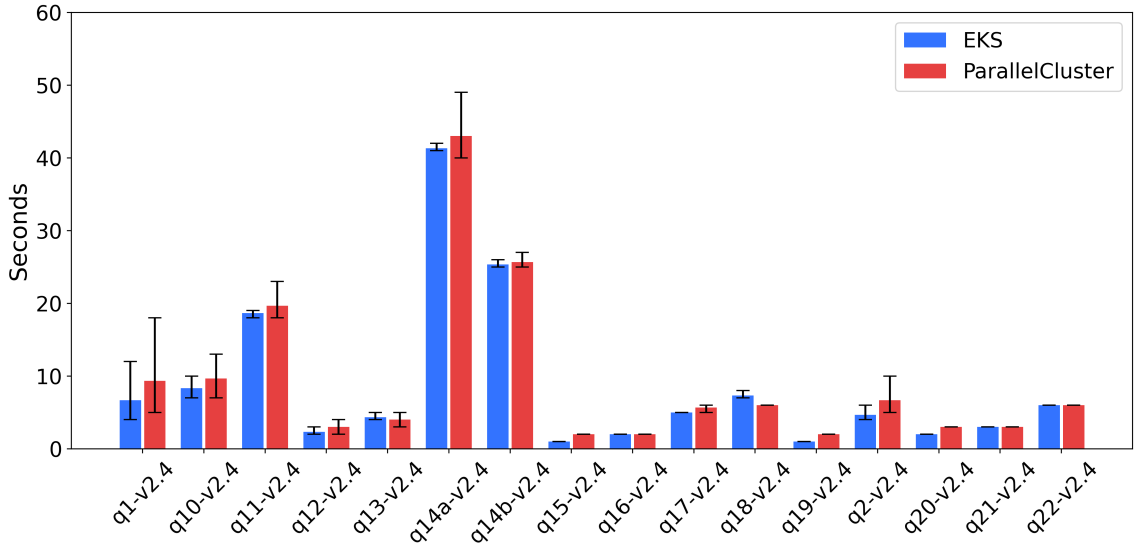


Figure 4: Evaluation using the TPC-DS benchmark.

First, we deploy the Spark Operator and MinIO via Helm. MinIO is an S3-compatible storage system, used to store the generated data. The benchmark requires a data generation phase before the actual submission of the workload. The benchmark YAMLs require that the S3 service is named `spark-k8s-data`, so this needs to be adjusted when deploying MinIO.

Once the supporting tools are ready, we submit the data generation and the benchmark `SparkApplications` in series, optionally adjusting the number of executors and the resource requirements of both driver and executor(s) as shown in Listing 1 (the example YAML uses 2 CPU cores for the driver and 3 executors, each occupying 1 CPU core). The same `SparkApplication` YAMLs, without any changes, run in both EKS and ParallelCluster/HPK.

Figure 4 shows the execution times for selected TPC-DS queries using a configuration with a single driver allocated 1 CPU and 14GB of memory, and a single executor also allocated 1 CPU and 14GB of memory. ParallelCluster/HPK demonstrates comparable efficiency to EKS, with most queries performing similarly. A few queries exhibit slightly higher execution times and greater variability, indicating that while HPK can handle Spark workloads effectively, performance consistency is somewhat lower compared to EKS.

```
apiVersion: "sparkoperator.k8s.io/v1beta2"
kind: SparkApplication
metadata:
  name: tpcds-benchmark-data-generation-1g
spec:
  ...
  driver:
    cores: 2
    memory: "8000m"
  executor:
    instances: 3
    cores: 1
    memory: "8000m"
    memoryOverhead: 2g
```

Listing 1: Controlling the number of executors to be deployed during the Spark TPC-DS data generation step.

4.2 Dask matrix multiplication

Dask is a Python library that enables parallel and distributed computing by decomposing large computations into smaller tasks executed concurrently across multiple cores or nodes. In Kubernetes environments, Dask typically runs as a distributed system composed of a scheduler and one or more worker pods. Both the scheduler and worker pods can be configured with specific CPU and memory resource requests.

First, we deploy the Dask cluster via Helm, providing a manifest that specifies the resource requests for both scheduler and worker pods. To evaluate the cluster's performance, we create a simple Python program that randomly generates two matrices of size `MATRIX_SIZE`, split into blocks of size `CHUNK_SIZE`, and multiplies them using Dask's distributed scheduler. This program is containerized into a Docker image and executed as a Kubernetes job, with `MATRIX_SIZE` and `CHUNK_SIZE` specified as environment variables to allow flexible configuration and reproducible benchmarking.

We tested four Dask cluster configurations (Table 1), varying the number of workers as well as CPU and memory allocations, on both EKS and ParallelCluster/HPK environments to evaluate their performance under different resource conditions. As illustrated in Figure 5, HPK performs comparably or better than native Kubernetes for medium-sized clusters, but under-performs in the edge cases involving very small or very large cluster configurations. The best-case scenario occurs with a medium-large-sized cluster, where HPK runs 19.4% faster than native, while the worst-case scenario is observed with the smallest cluster configuration, in which HPK runs 40.6% slower.

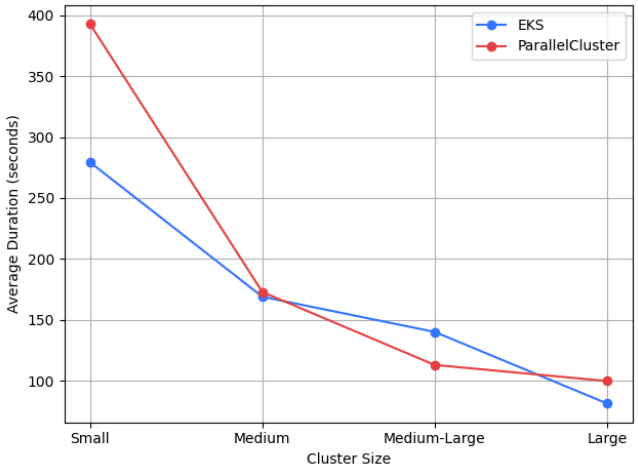


Figure 5: Evaluation using matrix multiplication with Dask for different cluster configurations.

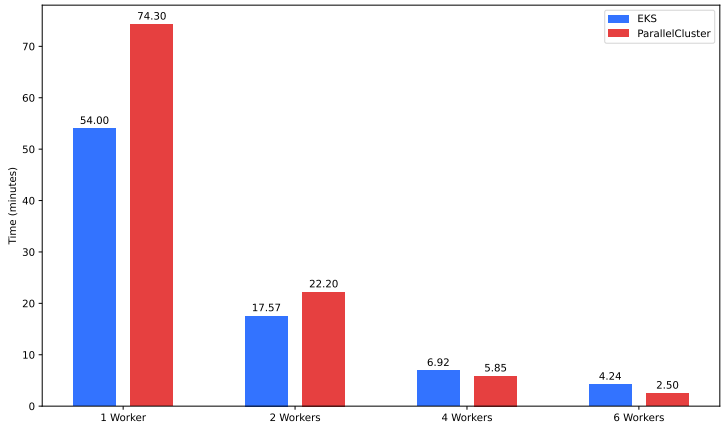


Figure 6: Evaluation using PyTorch on CPU-only nodes.

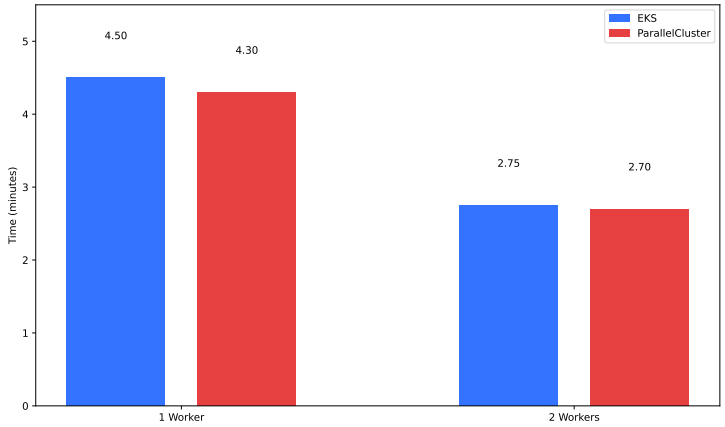


Figure 7: Evaluation using PyTorch on GPU nodes.

Table 1: Dask configurations used in evaluation.

Name	Driver conf.	# Workers	Worker conf.
Small	2 cores / 4 GiB	2	6 cores / 10 GiB
Medium	2 cores / 4 GiB	4	8 cores / 15 GiB
Medium-Large	2 cores / 4 GiB	5	12 cores / 16 GiB
Large	2 cores / 4 GiB	6	12 cores / 24 GiB

4.3 PyTorch model fine-tuning

We execute a Jupyter Notebook workflow [28]—part of Kubeflow Training Operator's examples—for fine-tuning a BERT language model for sentiment analysis, utilizing the Yelp review dataset as training data. The dataset subset contains 4,000 samples, with a 90/10 train–test split stratified by label. The training script is developed in the notebook and submitted to the Kubeflow Training Operator through the TrainingClient API. The operator then launches a `PyTorchJob`, configured to run for three epochs, with evaluation performed at the end of each epoch. The primary goal of each run is to measure the time required to complete fine-tuning in a resource-constrained environment, with the three-epoch execution running on both EKS and ParallelCluster/HPK. Figure 6 presents the results of the CPU-only runs, while Figure 7 shows the results when using GPUs. The nodes are the same (similar to c5.4xlarge nodes, but with 64 GiB RAM). To test scalability, each worker is limited to 4 vCPUs, 16 GiB RAM. Execution between CPU and GPU also differs in the selection of the PyTorch distributed communication backend; nccl when GPUs are available, else gloo. The evaluation reveals that HPK performs slower with a small number of workers. However, as the worker count increases, ParallelCluster/HPK outperforms EKS.

5 Conclusion

Kubernetes has become the industry standard runtime in the Cloud, providing the necessary abstractions to embrace the breadth and heterogeneity of available resources. Compatible Cloud-native tools are constantly evolving, covering a wide spectrum of applications, including database and queuing systems, interactive code execution frontends, workflow management utilities, as well as development frameworks that automatically optimize and scale operations. The ability to deploy this software directly on an HPC cluster via HPK opens up new possibilities for both Cloud and HPC users: it enables existing HPC users to tap on the vast collection of available Cloud applications and services, as stand-alone solutions or in hybrid computation scenarios combining Cloud frameworks with HPC codes. HPC centers may no longer need to maintain separate hardware partitions for Cloud analytics and HPC, as HPK allows running the Cloud workloads on the main HPC partition. Furthermore, HPK can be used to attract Cloud users to large HPC installations, offering them a familiar interface to seamlessly exploit the raw computing power available.

HPK runs as a user-triggered service, instantiated via Slurm. Users can submit Kubernetes workloads to their private API endpoint, which are effectively handled by the hpk-kubelet executable—a virtual Kubernetes node representing the entire cluster as a single entity that translates container lifecycle actions to Slurm scripts using Singularity/Apptainer commands. Evaluation using popular computation and machine learning frameworks suggests that HPK not only manages to run unmodified Cloud-native workloads on a Slurm-based environment, but also that—in several cases—this setup outperforms "vanilla" Kubernetes on the same hardware.

Acknowledgments

The authors thankfully acknowledge the support of the European Commission under the Horizon Europe Programme through projects DaFab (GA-101128693) and RISER (GA-101092993). AWS resources were provided by the National Infrastructures for Research and Technology (GRNET) and funded by the EU Recovery and Resiliency Facility. We are also grateful to our former colleagues Evangelos Maliaroudakis and Fotis Nikolaidis, who have realized most of the work done in earlier versions of HPK.

References

[1] 2025. Apptainer. https://apptainer.org
[2] 2025. Argo Workflows. https://argoproj.github.io/workflows/
[3] 2025. Cloud Native Computing Foundation. https://www.cncf.io
[4] 2025. Dask: Parallel Python fast and easy. https://dask.org/
[5] 2025. EKS Spark Benchmark: Benchmark results and best practice to run Spark workloads on EKS. https://github.com/aws-samples/eks-spark-benchmark
[6] 2025. Flannel. https://github.com/flannel-io/flannel
[7] 2025. HPK. https://github.com/CARV-ICS-FORTH/HPK
[8] 2025. Introducing SUNK: A Slurm on Kubernetes Implementation for HPC and Large Scale AI. https://coreweave.com/blog/sunk-slurm-on-kubernetes-implementations
[9] 2025. K3s: Lightweight Kubernetes. https://k3s.io
[10] 2025. Kubeflow. https://github.com/kubeflow/trainer
[11] 2025. The Kubernetes network model. https://kubernetes.io/docs/concepts/services-networking/#the-kubernetes-network-model
[12] 2025. Kubernetes Operator for Apache Spark. https://github.com/GoogleCloudPlatform/spark-on-k8s-operator
[13] 2025. Kubernetes: Production-Grade Container Orchestration. https://kubernetes.io
[14] 2025. MinIO. https://min.io/
[15] 2025. MPI Operator. https://github.com/kubeflow/mpi-operator
[16] 2025. OpenEBS: Kubernetes storage simplified. https://openebs.io/
[17] 2025. Slinky: Bridging Slurm And Kubernetes. https://www.schedmd.com/slinky/why-slinky/
[18] 2025. Slurm Bridge: Run Slurm as a Kubernetes scheduler. https://github.com/SlinkyProject/slurm-bridge
[19] 2025. slurm-operator: Kubernetes Operator for Slurm Clusters. https://github.com/SlinkyProject/slurm-operator
[20] 2025. Slurm Workload Manager. https://slurm.schedmd.com/documentation.html
[21] 2025. Soperator: Run Slurm in Kubernetes. https://github.com/nebius/soperator
[22] 2025. Sylabs: Singularity Container Technology & Services. https://sylabs.io
[23] 2025. Usernetes: Kubernetes without the root privileges. https://github.com/rootless-containers/usernetes
[24] 2025. Virtual-Kubelet. https://github.com/virtual-kubelet/virtual-kubelet
[25] Dong H. Ahn et al. 2018. Flux: Overcoming Scheduling Challenges for Exascale Workflows. In *2018 IEEE/ACM Workflows in Support of Large-Scale Science (WORKS)*. doi:10.1109/WORKS.2018.00007
[26] Antony Chazapis, Fotis Nikolaidis, Manolis Marazakis, and Angelos Bilas. 2023. Running Kubernetes Workloads on HPC. In *Proceedings of the International Workshop on Converged Computing on Edge, Cloud, and HPC* (Hamburg, Germany) *(WOCC'23)*. doi:10.1007/978-3-031-40843-4_14
[27] I. Colonnelli, B. Cantalupo, I. Merelli, and M. Aldinucci. 2021. StreamFlow: Cross-Breeding Cloud With HPC. *IEEE Transactions on Emerging Topics in Computing* 9, 04 (oct 2021), 1723–1737. doi:10.1109/TETC.2020.3019202
[28] Notebook: Fine-Tune BERT LLM for Sentiment Analysis with Kubeflow PyTorchJob. https://github.com/kubeflow/training-operator/blob/master/examples/pytorch/text-classification/Fine-Tune-BERT-LLM.ipynb.
[29] Gregory M. Kurtzer, Vanessa Sochat, and Michael W. Bauer. 2017. Singularity: Scientific containers for mobility of compute. *PLOS ONE* 12, 5 (05 2017), 1–20.
[30] Sergio López-Huguet, J. Damià Segrelles, Marek Kasztelnik, Marian Bubak, and Ignacio Blanquer. 2020. Seamlessly Managing HPC Workloads Through Kubernetes. In *High Performance Computing*, Heike Jagode, Hartwig Anzt, Guido Juckeland, and Hatem Ltaief (Eds.). Springer International Publishing, 310–320.

doi:10.1007/978-3-030-59851-8_20

[31] Boris Lublinsky, Elise Jennings, and Viktória Spišaková. 2022. A Kubernetes 'Bridge' operator between cloud and external resources. https://arxiv.org/abs/2207.02531v1

[32] Evangelos Maliaroudakis, Antony Chazapis, Alexandros Kanterakis, Manolis Marazakis, and Angelos Bilas. 2022. Interactive, Cloud-Native Workflows on HPC Using KNoC. In *Proceedings of the 5th Workshop on Interactive High-Performance Computing* (Hamburg, Germany) *(InteractiveHPC 2022)*. doi:10.1007/978-3-031-23220-6_15

[33] Tim Wickberg. 2022. Slurm and/or/vs Kubernetes. https://slurm.schedmd.com/SC22/Slurm-and-or-vs-Kubernetes.pdf

[34] George Zervas, Antony Chazapis, Yannis Sfakianakis, Christos Kozanitis, and Angelos Bilas. 2022. Virtual Clusters: Isolated, Containerized HPC Environments in Kubernetes. In *Proceedings of the 17th Workshop on Virtualization in High-Performance Cloud* (Hamburg, Germany) *(VHPC'22)*. doi:10.1007/978-3-031-23220-6_24

[35] Naweiluo Zhou, Yiannis Georgiou, Li Zhong, Huan Zhou, and Marcin Pospieszny. 2020. Container Orchestration on HPC Systems. In *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*. doi:10.1109/CLOUD49709.2020.00017